Neural architecture for ABRA — what the field built, what we built, and the gap

Written 2026-07-25. This is the design study that `docs/LITERATURE-v2.md` does not cover.

LITERATURE-v2 surveys the *algorithm* canon — CFR, DeepStack, ReBeL — and concludes correctly that the solution concept is a mixed Nash and the tractable method is depth-limited search with a learned leaf evaluator. It says almost nothing about the **network**: what to feed it, how to encode a battle, what shape the model should be. That gap is this document.

It is written against two constraints the project imposes: claims are measured rather than asserted, and negative results are reported plainly.

---

0. The one-paragraph summary

Both credible prior systems use **learned embeddings inside a Transformer over the whole battle**, and both treat the battle as a **sequence**, not a snapshot. ABRA's value model is an MLP over 121 hand-built numbers with **no history at all**. Our own ablation already showed representation matters 3.4x more than capacity, and the literature says exactly why: in a partially observed game the history *is* the hidden information, and we throw all of it away. The recommended next step is therefore **not a bigger network** — it is memory and embeddings, in that order.

---

1. What the field actually built

1.1 VGC-Bench (Angliss, Cui, Hu, Rahman, Stone — AAMAS 2026)

The only system targeting **VGC doubles**, which is our exact format family.

aspect	what they did
architecture	learned embeddings for moves/items/abilities, then a **3-layer Transformer encoder** with an aggregation token, attending over all **12 Pokémon** (both teams)
temporal	a **second Transformer along the time axis**, positional encoding, causal masking (frame stacking)
observation	`12 x (g + s + p)` — global (weather), side (screens), per-Pokémon features; mixed discrete and continuous
action space	**107 actions per Pokémon** — switches, moves x 3 targets, Tera. Team preview modelled as two joint "switch-in" actions
algorithm	PPO actor-critic; lr 1e-5, GAE lambda 0.95, clip 0.2, entropy 0.001, batch 64, 5,013,504 timesteps
data	**330k+ open-team-sheet games**, filtered to ladder rating >= 1200
result	beat a professional VGC competitor **in the single-team setting**

Their reported bottleneck, and it is the finding that matters most to us: performance degrades "considerably and consistently" as the number of training teams grows. The best single-team algorithm

"struggles at scaling up as team size grows" — degradation sets in **beyond 1-3 teams**.

1.2 Metamon (Grigsby et al., RLC 2025)

Pokémon **singles**, older generations, but the most complete scaling study that exists.

aspect	what they did
architecture	**causal Transformer** with actor and critic heads, at 15M / 50M / 200M parameters (RNNs of 500k-4M used for early tuning)
observation	**multimodal**: 87 text tokens of Pokémon vocabulary + 48 numerical values, fused by a Transformer encoder with summary tokens
memory	observations reveal **only the opponent's active Pokémon** — full team inference "relies entirely on memory"
action space	9 discrete actions (4 moves + 5 switches); singles, so no target selection
algorithm	offline RL: weighted BC + optional value term. IL / exponential-advantage / binary-filter / binary+MaxQ
data	**475k human battles -> ~950k POV trajectories, 38M timesteps** (2014-2024), plus **2M -> 5M self-play trajectories**
value	multiple discount factors trained in parallel, **gamma = 0.999** selected at test time; **two-hot classification** beat continuous regression
reward	binary win/loss with light shaping for damage dealt and health recovered

Their findings that transfer directly:

1. **RL beats pure BC**, but the specific RL variant barely matters — "little difference between the many RL variants considered." Effort spent choosing an algorithm is wasted; effort spent on data is not.
2. **Self-play data mattered more than human data.** Human-only: 41-58% GXE. With synthetic self-play: 64-80% GXE. This is the strongest external evidence that MEW's premise is sound.
3. **Memory is load-bearing**, not optional — longer context measurably improves play.
4. **Two-hot value classification** improved critic accuracy enough to change downstream behaviour.
5. Model-size scaling is "clearer for BC than for RL", with diminishing returns.
6. Their bottleneck was **dataset quality and diversity**, which is why they built the procedural "Variety Set" of 1k diverse teams per generation.

---

2. What ABRA built, measured against that

	VGC-Bench	Metamon	ABRA / PORY today
architecture	3-layer Transformer	causal Transformer, 15-200M	**MLP, 1 hidden layer, 64 units**
encoding	learned embeddings	87 text tokens + 48 numerics	**121 hand-built floats**
history	frame stacking + time-axis Transformer	full-battle causal context	**none — a single snapshot**
action space	107/Pokémon	9	**none; value only, no policy**
data	330k human	475k human + 5M self-play	11k ladder + 200k self-play
target	policy + value	policy + value	**P(win) only**

2.1 Our own ablation (measured, `engine/pory_nn.py`, 1.4M board states)

arm	log-loss
B2 `alive_diff + hp_diff` (the bar)	0.5258
L6 logistic, PORY's six features	0.5257
N6 network, material only	0.5227
LR logistic, rich (121 features)	0.5155
NR network, rich	**0.5154**

- nonlinearity alone: **0.0030**
- representation alone: **0.0102** (3.4x more)
- both: 0.0104 — i.e. nothing beyond representation

Caveat carried forward: that test set is ~86% self-play games from a fixed prior policy, so these absolute numbers are **not** comparable to the ladder-only run (B2 = 0.5734 there). The *ratio* between arms is the finding; the levels are not.

2.2 What this means

Our ablation and the literature agree, from opposite directions. We measured that capacity is not the constraint. The literature explains why: **we are missing the inputs that carry the information.**

---

3. The gap, in priority order

1. No memory. This is the big one. Pokémon is partially observed. What the opponent has revealed — which moves, which items, what they declined to do — is not in the current board, it is in the *history*. Metamon is explicit that full team inference "relies entirely on memory." PORY sees one frozen snapshot and cannot represent "they have not yet revealed their fourth Pokémon" or "they Protected last turn." No architecture change recovers information that was never fed in.

2. No embeddings. Both systems learn vector representations of species, moves, items, abilities, so that *similar things are near each other*. Our 121 features are hand-designed scalars: the model cannot learn that Incineroar and Arcanine play alike, because nothing in the input says so.

3. No policy head. PORY predicts P(win). It never predicts *what to do*. The depth-1 search we sketched needs a value function, so this is defensible for now — but a policy prior is what makes search tractable, and it is what both prior systems actually ship.

4. Doubles action structure is unmodelled. VGC-Bench's 107-action space with explicit target selection is the honest representation of doubles. Our measured branching factor (~100 options per side per turn, ~10,000 joint) is the same number arrived at independently — see `engine/mew_farm.js`.

---

4. The warning we should take seriously

VGC-Bench beat a professional **on one team** and degraded badly **beyond 1-3 teams**.

ABRA's self-play pool is **1,644 distinct teams**.

We are operating far inside the regime where the only published VGC system reports failure. This is not a reason to stop; it is a reason to be honest that a strong general VGC agent is an open research problem, and that our realistic near-term target is a **decision-support tool on a fixed team** — which is what a player actually wants anyway — rather than a general agent.

It also predicts our own result: with 1,644 teams and no team-identity representation, extra capacity has nothing to grip. That is exactly what the ablation showed.

---

5. Recommendation

In order, each cheap and independently checkable:

1. **Add history.** Feed the last *k* turns, not one snapshot. Cheapest possible version: concatenate the previous 3 board states plus a revealed-so-far bitmask. This is a feature-engineering change, not an architecture change, and the ablation predicts it is where the gain is.
2. **Add a revealed-information channel.** Explicitly encode what each side has shown: moves seen, items triggered, Pokémon still unrevealed. This is the hidden state, made visible.
3. **Two-hot value classification** instead of scalar regression. Metamon reports this improved critic accuracy materially; it is a ~20-line change to `pory_nn.py`.
4. **Only then** consider embeddings and a sequence model. Do not do this first — our own numbers say the return is 0.003 until 1-3 are done.
5. **Re-scope the goal** to fixed-team decision support, per section 4.

---

6. Licences and reuse

project	licence	reuse status
VGC-Bench (`cameronangliss/vgc-bench`)	check repo before any code reuse	**cite**; architecture ideas are freely usable, code is not vendored
Metamon (`UT-Austin-RPL/metamon`)	check repo before any code reuse	**cite**; datasets are Gens 1-4 singles, not applicable to Champions doubles

Neither project's data is directly usable here: VGC-Bench is Reg-G/H era SV VGC with open team sheets, Metamon is early-generation singles. **Champions is a different game** — different damage formula (SP stat system), different legality, different gimmick. Their *methods* transfer; their *data* does not. This is the same conclusion `docs/COMPETITORS.md` reached about dataset reuse.

Sources

- Angliss, Cui, Hu, Rahman, Stone. *VGC-Bench: Towards Mastering Diverse Team Strategies in Competitive Pokémon*. AAMAS 2026. arXiv:2506.10326 — https://arxiv.org/abs/2506.10326
- Code: https://github.com/cameronangliss/vgc-bench
- Grigsby et al. *Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers*. RLC 2025. arXiv:2504.04395 — https://arxiv.org/abs/2504.04395
- Code: https://github.com/UT-Austin-RPL/metamon